

Introduction to Machine Learning

K -means and Clustering Problems

Prof. Sundeep Rangan

1. You are given five data samples:

i	1	2	3	4	5
x_{i1}	0	1	0	2	2
x_{i2}	0	0	1	2	3

- (a) Draw the five points.
- (b) Starting with $K = 2$ cluster centers at $(0,0)$ and $(1,0)$, what are the cluster assignments and new cluster centers after one iteration of K -means?

Solution:

- (a) The five points are shown in Fig. 1.
- (b) The starting clusters are $\mu_1 = (0,0)$ and $\mu_2 = (1,0)$. The points closest to these cluster centers are:

$$C_1 = \{(0,0), (0,1)\},$$
$$C_2 = \{(1,0), (2,2), (2,3)\}.$$

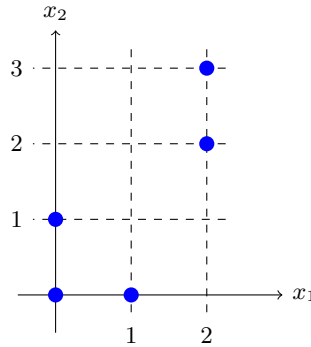


Figure 1: Data points for K -means clustering 1.

The new cluster centers are

$$\mu_1 = \frac{1}{2} [(0, 0) + (0, 1)] = (0, 0.5),$$
$$\mu_2 = \frac{1}{3} [(1, 0) + (2, 2) + (2, 3)] = \left(\frac{5}{3}, \frac{5}{3}\right).$$

2. *K-means for outlier detection.* Write a function for outlier detection:

```
def outlier_detect(Xtr,Xts,nc,t):  
    ...  
    return outlier
```

The function should:

- Perform *K*-means clustering on the training data `Xtr`;
- Given the matrix of test data `Xts`, it sets an output `outlier[i]=1` if the sample `Xts[i,:]` is greater than some distance `t` from all cluster centers.

Try to avoid for loops. You may assume you have the following functions:

```
km = KMeans(n_cluster=nc)    # Creates a K-Means object  
km.fit(X)    # Fits the k-means clusters  
km.cluster_centers_    # Returns the cluster centers  
                        # (one cluster per row)
```

Solution:

One solution to this problem is as follows:

```
def outlier_detect(Xtr,Xts,nc,t):  
    # Fit K-means on the training data  
    km = KMeans(n_cluster=nc)  
    km.fit(Xtr)  
  
    # Compute squared distance to the clusters  
    # dsq[i,j] = distance from x[i,:] to mu[j,:]  
    mu = km.cluster_centers_  
    dsq = np.sum((x[:,None,:] - mu[None,:,:])**2, axis=2)  
  
    # Find the minimum distance  
    dmin = np.min(dsq, axis=1)  
  
    # Declare an outlier if minimum distance  
    # exceeds threshold  
    outlier = (dmin > t**2)  
    return outlier
```

3. *Initialization.* Write a few lines of python code to initialize K -means by selecting K random samples of the training data as the cluster centers. Make sure you do not pick the same sample twice.

Solution:

The following function takes the training $\mathbf{x}$ and number of cluster centers nc and returns the initial cluster centers as random samples from $\mathbf{x}$.

```
def init(X,nc):
    # Get number of samples
    n = X.shape[0]

    # Randomly shuffle the indices from 0 to n-1
    # and then select the first nc values
    I = np.random.permutation(n)
    I = I[:nc]

    # Select the samples from the randomly chosen
    # indices
    mu = X[I:]

    return mu
```

4. *Clustering as pre-processing.* Suppose we want to cluster data and then fit a linear model in each cluster. You are given training data $\mathbf{x}_{tr}, \mathbf{y}_{tr}$ and test data $\mathbf{x}_{ts}, \mathbf{y}_{ts}$ for a regression problem. Write code to do the following:
- Perform K -means clustering on the training data $\mathbf{x}_{tr}$ with a given number nc clusters;
 - In each cluster in the training data, fit a linear model.
 - Compute the predicted outputs $\mathbf{y}_{hat,ts}$ and mean squared error of the model on the test data.

You may assume you have the following functions:

```
km = KMeans(n_cluster=nc)    # Creates a K-Means object
km.fit(X)                    # Fits the k-means clusters
km.predict(X)                 # Finds the index of the closest cluster

reg = LinearRegression()     # Creates a linear regression object
reg.fit(X,y)                  # Fits the linear model
yhat = reg.predict(X)         # Predicts the output
```

Note: You may need a list of regression objects.

Solution:

One solution is as follows:

```

# Fit clustering on training data
km = KMeans(n_clusters=nc)
km.fit(Xtr)

# Get cluster indices for training and test data
Itr = km.predict(Xtr)
Its = km.predict(Xts)

# Loop over clusters
reg = []
n = Xts.shape[0]
yhat_ts = np.zeros(n)
for i in range(nc):

    # Find training and test indices in the cluster
    Itri = np.where(Itr == i)[0]
    Itsi = np.where(Its == i)[0]

    # Fit the linear regression object in the cluster
    regi = LinearRegression()
    regi.fit(Xtr[Itri,:], ytr[Itri])

    # Predict on the test data
    yhati = regi.predict(Xts[Itsi,:])

    # Save in the vector
    yhat_ts[Itsi] = yhati

    # Save the regression object. You do not need to do this
    reg.append(regi)

# Compute MSE
mse = np.mean((yts-yhat_ts)**2)

```